

19 — Ward’s Method for Hierarchical Agglomerative Clustering, K-Means, Multidimensional Scaling, DBScan

Applied Statistics

Unat Teksen

April 20, 2026

1 Big Picture — What Is Clustering?

- **Unsupervised learning:** No labels are given. The data is just a set of points and we want to discover natural groupings on our own — as opposed to supervised learning where labels exist.
- **Formal setup:** Let $\tau = \{\underline{x}_i : i = 1, \dots, n\} \subseteq \mathbb{R}^p$ be the training set with a dissimilarity measure $d : \mathbb{R}^p \times \mathbb{R}^p \rightarrow [0, \infty)$ (not necessarily a metric).
- **Goal:** Find a partition of τ into k subsets $C_1, \dots, C_k$ such that:
 - $C_i \cap C_j = \emptyset \quad \forall i \neq j$ (non-overlapping)
 - $\bigcup_{j=1}^k C_j = \tau$ (full coverage)
- **Intuition:** Points within the same cluster should be similar; points in different clusters should be dissimilar.
- **Note:** There exist methods that allow overlapping clusters and units that are not clustered at all — e.g. **fuzzy clustering**. The formulation above is the standard hard partition case.
- **Methods covered:** Ward’s method (hierarchical), k -means / k -medoids (flat partition), DBSCAN (density-based), MDS (visualization / dimensionality reduction).

2 Ward’s Method for Hierarchical Agglomerative Clustering

2.1 Context and Goal

- **Hierarchical clustering** builds a **dendrogram** — a tree of nested clusters — by iteratively merging the two closest clusters (**agglomerative** = bottom-up).

- The question is: what does “closest” mean? Instead of deciding a linkage (single, complete, average), Ward’s method uses a different criterion: minimize the increase in **within-cluster spread** (ESS) at each merge.
- **Output:** A dendrogram where the y -axis shows the **ESS value** at each merge — not the distance. This is different from other linkage-based dendrograms.
- To obtain k clusters from the dendrogram: draw a horizontal line and cut it — wherever you cut, you get that many clusters.

2.2 Within-Cluster Sum of Squares (ESS)

- For a cluster C_j with centroid $\bar{x}_j$, the **within-cluster sum of squares** is:

$$ESS_j = \sum_{x_i \in C_j} (x_i - \bar{x}_j)^T (x_i - \bar{x}_j) = \sum_{x_i \in C_j} \|x_i - \bar{x}_j\|^2 \quad (1)$$

- The **centroid** of cluster C_k (mean of all points in the cluster):

$$\bar{x}_k = \frac{1}{\#C_j} \sum_{x_i \in C_j} x_i \quad j = 1, \dots, k \quad (2)$$

- The **total ESS** across all k clusters:

$$ESS = ESS_1 + ESS_2 + \dots + ESS_k \quad (3)$$

- **Intuition:** A tight, compact cluster has **small** ESS. A scattered, loose cluster has **large** ESS. ESS measures within-cluster variability.
- **Small example:** Cluster = $\{1, 3, 5\}$. Centroid = 3. $ESS = (1 - 3)^2 + (3 - 3)^2 + (5 - 3)^2 = 4 + 0 + 4 = 8$.

2.3 The Ward Algorithm

- **Start:** Each point is its own cluster (n clusters total).
- **At each step:** Find the two clusters whose merger causes the **minimum increase in total ESS** and merge them.
- **Stop:** When all points are in a single cluster.
- **Key difference from other methods:** We do *not* have a merge distance — we have ESS. On the dendrogram the y -axis shows the ESS value, not the distance.
- **Convergence:** Ward’s proceeds iteratively bottom-up; it is an agglomerative method.

2.4 Properties and Notes

- Ward’s method tends to produce **ellipsoidal, compact clusters**.
- It **avoids the chaining effect** that plagues single linkage: merging a chain would produce a huge ESS increase and would never be selected as the minimum-cost merge.

- **No chaining effect problem**, as opposed to complete linkage.
- Ward's method is **related to k -means**: both minimize within-cluster spread. Ward is hierarchical (builds a tree); k -means is flat (gives exactly k groups directly). Ward's method can also be seen as a variation leading toward k -means and k -medoids.

3 k -Means

3.1 Setup and Objective

- **Setting:** Let $\tau = \{\underline{x}_i : i = 1, \dots, n\} \subseteq \mathbb{R}^p$ and let $d : \mathbb{R}^p \times \mathbb{R}^p \rightarrow [0, \infty)$ be a dissimilarity measure (not necessarily a metric).
- **Given $k \geq 1$:** We need to choose k a-priori. Find k clusters $C_1, \dots, C_k \subseteq \tau$ (subsets of the training set) such that:

$$\begin{aligned} - C_i \cap C_j &= \emptyset \quad \forall i \neq j \\ - \bigcup_{j=1}^k C_j &= \tau \end{aligned}$$

- **Objective — minimize total within-cluster variability:**

$$\text{Find } C_1, \dots, C_k \text{ such that } \sum_{j=1}^k \sum_{\underline{x}_i \in C_j} d^2(\underline{x}_i, \bar{\underline{x}}_j) \text{ is minimised} \quad (4)$$

- **Centroid** of cluster C_j : the point in $\mathbb{R}^p$ minimizing the sum of squared distances to all members:

$$\bar{\underline{x}}_j = \arg \min_{\underline{x} \in \mathbb{R}^p} \sum_{\underline{x}_i \in C_j} d^2(\underline{x}_i, \underline{x}) \quad (5)$$

- **Note:** The centroid is the point that minimises the sum of all squared distances from that point to the cluster. If d is Euclidean, it is the arithmetic mean (barycentre):

$$\bar{\underline{x}}_j = \frac{1}{\#C_j} \sum_{\underline{x}_i \in C_j} \underline{x}_i \quad (6)$$

- The solution is found through an iterative procedure which usually finds a **local optimum** — global optimality is **not** guaranteed.

3.2 The k -Means Algorithm

- **Initialisation (two options):**
 - Randomly assign all training points among the k clusters $C_1, \dots, C_k$, then go to **Step 1**.
 - Randomly pick k centroids $\bar{\underline{x}}_1, \dots, \bar{\underline{x}}_k \in \mathbb{R}^p$, then go to **Step 2**.
- **Important:** The final output depends strongly on this random initialisation — so each time we run k -means we may get a different result. We need to repeat the algorithm a few times to be sure the result is robust.

- **Iterate until convergence:**

- **Step 1 — Update centroids:** For $j = 1, \dots, k$, compute the centroid $\bar{x}_j$ of the current cluster C_j . After this step we have updated $\bar{x}_1, \dots, \bar{x}_k$.
- **Step 2 — Reassign points:** For all $x_i \in \tau$, assign each unit to the cluster C_j whose centroid is closest:

$$x_i \in C_j \quad \text{if} \quad d^2(x_i, \bar{x}_j) \leq \min\{d^2(x_i, \bar{x}_l) : l = 1, \dots, k\} \quad (7)$$

- **Convergence:** Stop when the centroids at **Step 1** are the same as those computed in the previous iteration — nothing changes.
- **Convergence guarantee:** With Euclidean distance, convergence is proved. For a general d that is not Euclidean we may iterate forever — otherwise we need to prove it (very hard). In practice: try it and fingers crossed.
- **Note on Step 1 (the difficult step):** An easy way out is to minimize only with respect to the training set τ and not all of $\mathbb{R}^p$:

$$\bar{x}_j = \arg \min_{x \in \tau} \sum_{x_i \in C_j} d^2(x_i, x) \quad j = 1, \dots, k \quad (8)$$

In this case $\bar{x}_j$ is called a **medoid** and the algorithm is **k -medoids**.

3.3 k -Medoids Variant

- The medoid is the training point most central to the cluster — it is a real observed data point, not a computed average.
- **Advantage:** Only a finite number of candidates to check; clusters are defined by actual observed units.
- **Advantage:** In this case we need to compute only a finite number of steps and the clusters will be specified from units that we have observed and not on the centroid.
- **Note:** The mean will be the average that nobody knows since we minimize in τ and not in $\mathbb{R}^p$ — indeed there is no such thing as the average man (or average city).
- **Example:** Cities as data. There is no city at the average coordinates — but there is a real city closest to all others in the cluster. That city is the medoid.
- **Disadvantage:** Slower than k -means for large datasets.

3.4 Choosing k : The Elbow Method

- Run the algorithm for different values of k and compute:

$$W(k) = \sum_{j=1}^k \sum_{x_i \in C_j} d^2(x_i, \bar{x}_j) \quad (9)$$

- Plot $W(k)$ against k :
 - When $k = n$: each point is its own cluster $\Rightarrow W(k) = 0$.

- As k decreases: $W(k)$ increases.
- Look for an **elbow** — a sharp bend in the curve — and choose k there.
- **Principle:** The less k is, the easier it is to explain what is going on. So we want the smallest k that still captures structure.

3.5 Additional Notes on k -Means

- **When data are not points:** When working with functions, tensors, or images, clustering can mostly only be done with k -means (hierarchical methods become very difficult).
- **Kernel trick:** We can work with different features — take the data, transform it (e.g. via kernel methods), and then apply k -means in the transformed space.
- **Many clustering methods exist** but the key problem is deciding the similarity measure.
- **Warning — PCA before clustering:** There is no obvious link between principal components and clusters. It may be harmful to do PCA first and then cluster. The same holds for PCA and classification. Principal components capture directions of maximum variance, which need not align with cluster structure.

4 Graphical Representations and Dimensionality Reduction

4.1 Motivation

- We would like to reduce the dimensionality so that we can spot clusters more easily.
- **Two main approaches:**
 - **Fisher's Scores after LDA:** After clustering we have labels, so we can use LDA and then take Fisher's Scores to project the data.
 - **Multi-Dimensional Scaling (MDS):** A technique for both clustering support and graphical representation — covered in the next section.

5 Multidimensional Scaling (MDS)

5.1 Goal and Setup

- **Problem:** Data lives in $\mathbb{R}^p$ with p very large — we cannot visualize it directly.
- **Setting:** We have training set $\tau = \{\underline{x}_i : i = 1, \dots, n\} \subset \mathbb{R}^p$ and a distance d . We want to represent the problem in $\mathbb{R}^q$ with $q < p$ (for graphical reasons it is nice if $q = 1, 2, 3$) using Euclidean distance d_e .
- **Goal:** Change the dataset to a new one $\tau' = \{\underline{y}_i : i = 1, \dots, n\}$ with $\underline{y}_1, \dots, \underline{y}_n$ such that:

$$d_e(\underline{y}_i, \underline{y}_j) \approx d(\underline{x}_i, \underline{x}_j) \quad (10)$$

- The new embedding:

$$\tilde{\mathbf{X}} = \begin{bmatrix} \underline{y}_1^T \\ \vdots \\ \underline{y}_n^T \end{bmatrix} \in \mathbb{R}^q, \quad q \ll p, \quad d_{ij} \simeq \delta_{ij} \quad (11)$$

where $d_{ij} = d(\underline{x}_i, \underline{x}_j)$ and $\delta_{ij} = \delta(\underline{y}_i, \underline{y}_j)$ (Euclidean in new space).

- **Note:** What do we mean by “close”? It needs to be specified — this is where the choice of objective function comes in.
- **Practical use:** After embedding into $\mathbb{R}^2$ or $\mathbb{R}^3$ you can plot the data and visually identify clusters.
- **Example:** We have different cities in the world. As the distance between two cities we can take fly time — cities are on a sphere so Euclidean distance would not be good. Moreover fly time could be asymmetric because of wind streams. Suppose we render flight time symmetric: we then want a map in $\mathbb{R}^2$ with all the cities where the distance between the points is as close as possible to the fly time.

5.2 Non-Uniqueness

- The solution is **not unique**: in Euclidean space, any rigid transformation (rotation, reflection, translation) preserves Euclidean distances and gives another valid solution.
- The first solution is not very evocative — we need to try it many times.
- We can reflect, rotate, and translate to get a good interpretable result.

5.3 Objective Functions

- Let $D = [d_{ij}]$ be the original distance matrix and $\Delta = [\delta_{ij}]$ the Euclidean distance matrix of new points. The objective is to find $y_1, \dots, y_n$ such that:

$$\sum_{i=1}^n \sum_{j=1}^n (d_{ij} - \delta_{ij})^2 \quad \text{is minimum} \quad (12)$$

- **Classical MDS:** Minimize the sum of squared deviations:

$$\sum_{i \neq j} (d_{ij} - \delta_{ij})^2 \quad (13)$$

- **Modern MDS — Kruskal et al.:** Minimize the **STRESS** function:

$$STRESS = \frac{\sum_{i \neq j} (\theta(d_{ij}) - \delta_{ij})^2}{\sum_{i \neq j} \delta_{ij}^2} \quad (14)$$

where $\theta : \mathbb{R} \rightarrow \mathbb{R}$ is a **monotone increasing function** that is also optimized over.

- Equivalently (textbook form with $\theta : [0, \infty) \rightarrow [0, \infty)$):

$$STRESS = \frac{\sum_{i=1}^n \sum_{j=1}^n (\theta \cdot d_{ij} - \delta_{ij})^2}{\sum_{i=1}^n \sum_{j=1}^n \delta_{ij}^2} \quad (15)$$

- We want to minimize STRESS both with respect to τ' (the new positions $\underline{y}_i$) and with respect to θ .
- **Intuition:** θ allows preserving only the *ordering* (rank) of distances, not exact values. “A is closer to B than to C” is preserved; the exact values need not match.
- We evaluate only θ in a certain class — otherwise there are an infinite number of different θ .
- **Note:** There is no theorem guaranteeing that we get the right thing — sometimes it works.
- **Note:** There are people who in the STRESS take the fourth power instead of the square, and so on.

5.4 Choosing q : The STRESS Scree Plot

- Compute $STRESS(q)$ for $q = 1, 2, 3, \dots$ and plot against q .
- Choose q at the **elbow** of the STRESS scree plot — the same idea as the elbow method for k -means and as PCA’s scree plot.
- We stop at an elbow.

5.5 Special Case: Euclidean Distance

- If $d = d_e$ (Euclidean), then MDS reduces exactly to **PCA**:
 - The linear subspace of dimension q closest to the original training set is given by the PCA — it is given by the space spanned by stopping at the first q principal components.
 - So if d is Euclidean, we don’t use MDS but just apply PCA.
- **Computational comparison:**
 - To perform PCA we need to decompose a $p \times p$ matrix.
 - To perform MDS we need to decompose an $n \times n$ matrix.
 - Use whichever matrix is smaller.
- **Remark:** If d is Euclidean, $y_1, \dots, y_n$ are represented by the scores of principal components $PC_1, \dots, PC_q$.
- **Non-uniqueness remark:** The representation is not unique — we have to find the one that fits our purposes since there is an equivalence class of solutions.

5.6 MDS for Clustering

- Moreover, MDS can be used in principle for clustering: we use MDS and then we have a training set in $\mathbb{R}^q$ with d_e , so we can then apply any clustering technique.
- MDS offers us the **transformation of the features**.
- Useful when the original d is non-Euclidean or when the data has no natural vector representation.

6 DBScan — Density-Based Spatial Clustering

6.1 Motivation

- k -means forces every point into a cluster and requires k in advance.
- **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise) identifies clusters as **dense regions** in the data and explicitly marks sparse, isolated points as **noise** — no forced assignment.
- Besides finding clusters, this algorithm also finds **outliers**, letting points without a high density go without being assigned to any group.
- **Two hyperparameters:** $\epsilon > 0$ (neighborhood radius) and $MinPts \in \mathbb{N}$ (minimum number of neighbors).

6.2 Definition 1: Neighborhood

- For $\epsilon > 0$, the **ϵ -neighborhood** of $\underline{x}$ is:

$$N_\epsilon(\underline{x}) = \{\underline{y} \in \mathbb{R}^p : d(\underline{x}, \underline{y}) \leq \epsilon\} \quad (16)$$

- **Remark:** $|N_\epsilon(\underline{x})|$ is the size of the neighborhood — the number of units in $\mathbb{X}$ belonging to $N_\epsilon(\underline{x})$.

6.3 Definition 2: Point Classes

Given a fixed integer $MinPts$, each point $\underline{x}_i \in \mathbb{X}$ is classified as one of:

- **Core Point:** $|N_\epsilon(\underline{x}_i)| \geq MinPts$. The point sits in a dense region — it has enough neighbors.
- **Border Point:** $\underline{x}_i$ is not a core point, but belongs to $N_\epsilon(\underline{x}_j)$ for some core point $\underline{x}_j$.
- **Noise Point:** If it is neither a core point nor a border point. Isolated — an outlier.

6.4 Definition 3: Directly Density Reachable

- $\underline{x}_j \in \mathbb{X}$ is **directly density reachable** from $\underline{x}_i$ if:
 - $\underline{x}_i$ is a core point, and
 - $\underline{x}_j \in N_\epsilon(\underline{x}_i)$

- **Intuition:** A core point can “reach” any of its neighbors in one hop.
- **Note:** This relation is **asymmetric** — $\underline{x}_j$ reachable from $\underline{x}_i$ does not imply $\underline{x}_i$ reachable from $\underline{x}_j$ (because $\underline{x}_j$ might not itself be a core point).

6.5 Definition 4: Density Reachable

- $\underline{x}_j \in \mathbb{X}$ is **density reachable** from $\underline{x}_i$ if there exists a finite sequence $\underline{y}_1, \dots, \underline{y}_q$ such that:
 - $\underline{y}_1 = \underline{x}_i, \quad \underline{y}_q = \underline{x}_j$
 - $\underline{y}_{l+1}$ is directly density reachable from $\underline{y}_l \quad \forall l = 1, \dots, q-1$
- **Remark:** All intermediate points $\underline{y}_1, \dots, \underline{y}_{q-1}$ need to be core points.
- **Remark:** (Directly) Density reachable is **not a symmetric property**.
- **Intuition:** Multi-hop path through a chain of core points.

6.6 Definition 5: Density Connected

- Let $\underline{x}_i, \underline{x}_j \in \mathbb{X}$. They are **density connected** if there exists $\underline{x} \in \mathbb{X}$ such that both $\underline{x}_i$ and $\underline{x}_j$ are density reachable from $\underline{x}$.
- **Note:** This relation is **symmetric** — unlike density reachability.

6.7 What DBScan Finds

- DBSCAN identifies subsets $C \subseteq \mathbb{X}$ (clusters) such that:
 - If $\underline{x}_i \in C$ and $\underline{x}_j$ is density reachable from $\underline{x}_i$, then $\underline{x}_j \in C$.
 - If $\underline{x}_i, \underline{x}_j \in C$, then $\underline{x}_i$ and $\underline{x}_j$ are density connected.
- Points not assigned to any cluster are labeled **noise**.
- **Remark:** Some border points may belong to two different clusters — the clusters are therefore **not necessarily a strict partition** of $\mathbb{X}$.

6.8 Curse of Dimensionality

- The obvious problem that appears is the **curse of dimensionality**: as p grows, all points become approximately equidistant.
- Even if ϵ is too large, points will be too far from others.
- Tuning ϵ and $MinPts$ becomes very difficult in high dimensions.
- DBSCAN is most effective in low to moderate dimensions.

7 Method Comparison

Method	Need k ?	Finds noise?	Cluster shape	Output
Ward	No	No	Ellipsoidal	Dendrogram
k -means	Yes $\triangle$	No	Spherical	k clusters
k -medoids	Yes $\triangle$	No	Flexible	k clusters
DBSCAN	No $\checkmark$	Yes $\checkmark$	Any shape	Clusters + noise
MDS	—	—	—	2D/3D embedding

- MDS is not a clustering algorithm — it is a **visualization and preprocessing tool** used alongside other methods.
- **When to use each method:**
 - **Ward:** No idea how many clusters; want a tree to explore and then cut.
 - **k -means:** Want exactly k compact clusters quickly; run multiple times for robustness.
 - **k -medoids:** Data are objects/cities; need a real observed center point; non-Euclidean d .
 - **DBSCAN:** Clusters have irregular shapes; some points are outliers; k unknown.
 - **MDS:** Data is high-dimensional; want to visualize or work with a non-Euclidean d before clustering.

Türkçe Özet — Basit Anlatım

Büyük Resim

- **Kümeleme (clustering):** Etiket yok, sadece veri noktaları var. Amacımız bu noktaları doğal gruplara ayırmak — gözetimsiz öğrenme (unsupervised learning).
- **Gözetimli öğrenmeden farkı:** Gözetimli öğrenmede etiketler vardır (“bu e-posta spam”). Kümelemede kimse bize grupları söylemez — veri kendi söyler.
- **Örnek:** 1000 müşteri, sadece yaş ve harcama bilgisi var. Kümeleme ile “genç-az harcayan”, “orta yaşlı-çok harcayan” gibi grupları *kendiliğinden* bulmak istiyoruz.
- **Örtüşen kümeler:** Bazı yöntemler (fuzzy clustering gibi) kümelerin örtüşmesine veya bazı noktaların hiç atanmamasına izin verir. Standart yöntemlerde her nokta tam olarak bir kümeye aittir.

Ward Yöntemi

- **Ne yapıyor?** Noktaları ağaç yapısında (dendrogram) birleştiriyor — alttan üste, her adımda toplam ESS’yi en az artıran iki kümeyi birleştiriyor.
- **ESS nedir?** Her noktanın kendi küme merkezine uzaklığının karelerinin toplamı. Küme ne kadar sıkıysa ESS o kadar küçük.

$$ESS_j = \sum_{x_i \in C_j} \|x_i - \bar{x}_j\|^2$$

- **Küçük örnek:** Küme = {1, 3, 5}, merkez = 3, $ESS = 4 + 0 + 4 = 8$.
- **Dendrogramı nasıl kullanırsın?** Yatay bir çizgiyle kesersen — nerede kesersen o kadar küme elde edersin.
- **y-ekseni farkı:** Diğer hiyerarşik yöntemlerde *y*-ekseninde birleşme *mesafesi* gösterilir. Ward’da ise *y*-ekseninde *ESS değeri* gösterilir.
- **Zincirleme efekti yok:** Tek bağlantı (single linkage) yönteminde kümeler uzun zincirler gibi uzayabilir. Ward’da her adımda en kompakt birleşimi seçtiğin için bu sorun yaşanmaz. Complete linkage’ın aksine zincirleme efekti problemi yok.
- **k-means ile bağlantısı:** Her ikisi de küme içi yayılımı (ESS) minimize eder. Ward hiyerarşik (ağaç); *k*-means düz (direkt *k* küme). Ward, *k*-means ve *k*-medoid’e giden bir basamak olarak da görülebilir.

k-Means

- **Ne yapıyor?** Tam olarak *k* kümeye böler. Amacı her noktanın kendi küme merkezine uzaklığının karesini minimize etmek:

$$W(k) = \sum_{j=1}^k \sum_{x_i \in C_j} d^2(x_i, \bar{x}_j)$$

- **Centroid (merkez) nedir?** Kümedeki tüm noktalara olan kare uzaklıkların toplamını minimize eden noktadır. Öklid uzaklığı kullanılıyorsa bu sadece aritmetik ortalamadır.
- **Algoritma:** Başlangıçta rastgele merkezler seç. Sonra iki adımı dönüşümlü tekrarla: (1) her kümenin yeni merkezini hesapla, (2) her noktayı en yakın merkeze ata. Merkezler değişmeyince dur.
- **Dikkat — rastgele başlangıç:** Her çalıştırmada farklı sonuç verebilir. Algoritmayı birkaç kez çalıştır, en iyi $W(k)$ 'yı al.
- **Yakınsama:** Öklid uzaklığı için yakınsama ispatlanmıştır. Genel bir d için garanti yok — dene ve şansına bak.
- **Yerel optimum:** Algoritma genellikle yerel bir optimum bulur — global optimum garanti değildir.
- **k -medoid farkı:** Normal k -means'te merkez, kümedeki noktaların ortalamasıdır — bu nokta gerçek bir veri noktası olmayabilir. k -medoid'te ise merkez gerçek bir veri noktası olmak zorundadır. “İki şehrin ortalaması” diye bir şehir yoktur — ama herkese en yakın gerçek şehir vardır, işte o medoid olur. Ayrıca $\mathbb{R}^p$ 'de değil sadece τ üzerinde minimize edildiğinden adım sayısı sonludur.
- **Doğru k 'yı seçmek — Dirsek Yöntemi:** $W(k)$ 'yı k 'ya karşı çiz. $k = n$ olunca $W = 0$. Eğride keskin bir *dirsek* (kırılma noktası) ararız ve oradaki k 'yı seçeriz. k ne kadar küçükse sonucu açıklamak o kadar kolaydır.
- **Fonksiyon/tensör/görüntü verisi:** Noktalar yerine fonksiyon veya görüntülerle çalışıyorsan hiyerarşik yöntemler zorlaşır — k -means hala çalışır çünkü sadece noktalar arası uzaklık gerekir.
- **Kernel trick:** Veriyi önce dönüştür (kernel yöntemi), sonra k -means uygula.
- **Uyarı — PCA sonrası kümeleme:** Ana bileşenler ile kümeler arasında doğrudan bir bağlantı yoktur. Önce PCA, sonra kümeleme yapmak sonucu bozabilir. Aynı uyarı PCA sonrası sınıflandırma için de geçerlidir.

MDS — Çok Boyutlu Ölçekleme

- **Problem:** Verimiz çok yüksek boyutlu. Bunu görselleştiremeyiz — insan beyni en fazla 3 boyutu algılayabilir.
- **Amaç:** Yüksek boyutlu veriyi 2 ya da 3 boyuta indirgiyoruz ama noktalar arasındaki uzaklıkları mümkün olduğunca koruyoruz: $d_e(\underline{y}_i, \underline{y}_j) \approx d(\underline{x}_i, \underline{x}_j)$.
- **“Yakın” ne demek?** Bu belirtilmeli — amaç fonksiyonu seçimiyle tanımlanır.
- **Somut örnek:** Şehirler arasındaki uçuş sürelerini biliyorsun. Şehirler bir küre üzerinde, Öklid uzaklığı iyi olmaz. Üstelik uçuş süreleri rüzgar akımları yüzünden asimetrik olabilir. Önce simetrikleştirebiliriz, sonra MDS ile 2D harita çıkar — birbirine yakın uçuş süresine sahip şehirler haritada da yakın çıkar.
- **Çözüm tekil değildir:** Öklid uzayında döndürme, yansıma, öteleme gibi katı dönüşümler mesafeleri korur — yani pek çok geçerli çözüm vardır. Bu yüzden algoritmayı birkaç kez çalıştırıp en yorumlanabilir sonucu seçmek gerekir.

- **Klasik MDS:** $\sum_{i \neq j} (d_{ij} - \delta_{ij})^2$ minimize edilir.
- **Modern MDS (Kruskal):** STRESS minimize edilir. θ fonksiyonu sadece uzaklıkların *sıralamasını* korumaya çalışır — tam değerlerini değil. “A, B’ye C’den daha yakın” sıralaması korunur.
- **θ fonksiyonu kısıtı:** Sonsuz sayıda θ olmaması için belirli bir fonksiyon sınıfıyla kısıtlanır.
- **Doğru q ’yu seçmek:** STRESS’i q ’ya karşı çiz, dirsekte kes. k -means’teki dirsek yöntemiyle aynı mantık.
- **Teorik garanti yok:** Doğru sonucu garanti eden bir teorem yoktur — bazen çalışır. Bazı varyantlarda STRESS’te karenin yerine dördüncü kuvvet kullanılır.
- **Özel durum — Öklid uzaklığı:** Orijinal uzaklık zaten Öklid ise MDS = PCA. İlk q ana bileşen skorları tam olarak MDS çıktısını verir. Bu durumda MDS yapmana gerek yok, PCA uygula.
- **Hesaplama karşılaştırması:** PCA $p \times p$ matris ayrıştırır; MDS $n \times n$ matris ayrıştırır. Hangisi küçükse onu kullan.
- **Kümeleme için kullanım:** MDS sonrası $\mathbb{R}^q$ ’da Öklid uzaklığıyla verin var — istediğin kümeleme yöntemini uygulayabilirsin. MDS bize **özelliklerin dönüşümünü** sağlar.

DBSCAN

- **Temel fikir:** k -means’ten farklı olarak k ’yı önceden bilmene gerek yok. Yoğun bölgeleri küme sayar, seyrek bölgedeki noktaları *gürültü* (noise/outlier) olarak işaretler — hiçbir kümeye zorla atama yapmaz.
- **İki parametre:** ϵ (komşuluk yarıçapı) ve $MinPts$ (minimum komşu sayısı).
- **Üç tip nokta:**
 - **Çekirdek nokta:** ϵ çemberinde en az $MinPts$ komşusu var. Yoğun bölgenin merkezi.
 - **Sınır nokta:** Kendisi yoğun değil ama bir çekirdek noktanın ϵ çemberinin içinde.
 - **Gürültü noktası:** Ne çekirdek ne sınır. Hiçbir kümeye ait değil — aykırı değer (outlier).
- **Doğrudan yoğunluk erişilebilir:** $\underline{x}_j, \underline{x}_i$ ’den direkt erişilebilir $\Leftrightarrow \underline{x}_i$ çekirdek ve $\underline{x}_j \in N_\epsilon(\underline{x}_i)$. Tek adımlık atlama. Bu ilişki *simetrik değildir*.
- **Yoğunluk erişilebilir:** Çekirdek noktalar zinciriyle çok adımda erişilebilir. Tüm ara noktalar çekirdek olmak zorundadır. Bu da *simetrik değildir*.
- **Yoğunluk bağlantılı:** İkisi de ortak bir üçüncü noktadan erişilebiliyorsa bağlantılıdır. Bu ilişki *simetrik*dir.
- **Sınır noktası uyarısı:** Bazı sınır noktaları iki farklı kümenin komşuluğuna girebilir — bu yüzden DBSCAN katı bir bölüm (partition) üretmez, kümeler örtüşebilir.

- **Somut örnek:** Şehrin farklı bölgelerindeki kafe konumları. Kafeler yoğun mahallelerde kümeler oluşturur. Tek başına duran uzak bir kafe gürültü noktasıdır.
- **Boyutsal lanet:** Boyut arttıkça tüm noktalar birbirinden eşit uzaklıkta olmaya başlar. ϵ ayarı imkânsızlaşır — DBSCAN düşük ve orta boyutlarda çok daha iyi çalışır.
- **Avantajları:** k 'yı bilmene gerek yok; kümelerin şekli ne olursa olsun (yıldız, halka, herhangi bir şekil) bulabilir; outlier'ları tespit eder.